

Chapter 1

Background and Related Work

This chapter establishes the context, motivation, and the core knowledge that the thesis relies upon. Section 1.1 first delimits the scope of the web intrusion detection problem that the system targets: its input data, its expected output, and the classes of attacks that fall inside and outside the processing boundary. Section 1.2 then surveys representative studies that address attack detection on the same HTTP dataset, analysing their strengths and weaknesses in order to expose the research gap and the motivation behind this work. The remaining sections present the foundational knowledge most tightly connected to the system: the notion of an intrusion detection system and its two detection paradigms (Section 1.3), common web application attacks (Section 1.4), signature-based detection with regular expressions (Section 1.5), supervised machine learning for classification (Section 1.6), unsupervised anomaly detection following the one-class learning principle (Section 1.7), and finally the feature extraction technique together with the evaluation metrics used throughout the thesis (Section 1.8). These topics are the direct basis for the three-tier hybrid architecture realised in the subsequent chapters.

1.1 Scope of Research

This thesis focuses on the problem of *web intrusion detection* at the application layer through the analysis of access logs produced by a web server. The task is framed as a binary classification problem: for every recorded HTTP/HTTPS request, the system must decide whether the request is *benign* (clean) or an *attack*, and it must additionally rank the severity of each alert so that a security analyst can prioritise the investigation.

Regarding the input data, the system operates directly on the widely used Apache log formats, namely the Combined Log Format and the Common Log Format. Each log line carries fields such as the source IP address, the timestamp, the request line (method, URL, and protocol version), the returned status code, the **Referer** header, and the **User-Agent** header. During implementation, the log parsing module (`parse_log_entry` in `apache_log.py`) decomposes the request line into method, path, query string, and parameters, and it iteratively URL-decodes the request to recover the original payload before feeding it to the detection tiers. A noteworthy characteristic is that the body of POST requests is also preserved and appended to the URL as a `POST_BODY` parameter, so that parameter-analysis techniques apply to attacks delivered through the request body and not merely through the query string.

Regarding the output, for every log line the system produces a structured record (in JSON Lines format) containing the `is_attack` verdict, an aggregated threat score,

a severity level (CRITICAL, HIGH, MEDIUM, LOW), and the list of detection tiers that fired. The alerts are aggregated into a visual dashboard and a live alert file for real-time monitoring.

The classes of attacks addressed by the thesis are application-layer web attacks that manifest directly in the content of the HTTP request. Specifically, the system targets the following attack groups:

- (i) SQL Injection;
- (ii) Cross-Site Scripting (XSS);
- (iii) Path Traversal and Local File Inclusion;
- (iv) Operating-system Command Injection;
- (v) Remote File Inclusion;
- (vi) Access to sensitive configuration or backup files;
- (vii) Use of automated scanning tools identified through the **User-Agent** header;
- (viii) Parameter Tampering and HTTP Method Tampering.

Conversely, several topics lie outside the scope of this thesis. The system does not handle network- or transport-layer attacks such as volumetric distributed denial of service, it does not perform packet-level decryption of TLS traffic (it works on logs already decrypted by the server), and it does not assume the blocking role of a Web Application Firewall. The emphasis of the thesis is on *detection* and *alerting*, which form the prerequisite for subsequent response measures.

For training and evaluation, the thesis builds a balanced dataset of 111,065 log lines, of which one half is attack traffic and the other half is benign traffic. This dataset combines three sources: the standard HTTP DATASET CSIC 2010 [?], real attack samples extracted from the open-source PayloadsAllTheThings repository, and synthetic traffic generated automatically to increase the diversity of encoding variants. Choosing CSIC 2010 as the backbone of the dataset allows the results of this thesis to be compared directly with the related studies presented in the next section, since that dataset is also the one used by most of those studies.

1.2 Related Work

Enhancing intrusion detection with machine learning has been studied extensively. This section reviews representative studies that operate on the HTTP CSIC 2010 dataset, analyses the strengths and weaknesses of each approach, and thereby identifies the gap that this thesis aims to fill.

1.2.1 Comparing machine learning algorithms on CSIC 2010 (Pham et al., 2016)

Pham, Hoang, and Vu [?] surveyed and compared five machine learning algorithms—Random Forest, Logistic Regression, Decision Tree, AdaBoost, and Stochastic Gradient Descent—on the CSIC 2010 dataset. The strength of their approach lies in the feature

extraction step applied to the raw text of the HTTP request: the authors combine three techniques, namely tokenizing, occurrence counting, and re-weighting by the inverse document frequency (TF-IDF), producing a sparse matrix as input to the classifier. The experimental results show that Logistic Regression achieved the best performance, with an F1 score of about 96.11% for the attack class; after tuning the inverse regularisation strength (choosing $C = 13.0$) together with recursive feature elimination (retaining 77,558 features), the precision and recall were further improved.

Nevertheless, this approach exhibits several limitations. First, the bag-of-words representation with tens of thousands of features depends heavily on the vocabulary learned from the training set, which makes the model bulky and prone to losing validity when faced with traffic from a different web application that uses a different vocabulary. Second, this is a purely *single-model supervised* approach that requires fully labelled data and, by nature, only learns the characteristics of attacks already seen; the authors themselves acknowledge in their conclusion that evaluation on unseen attacks remains open. Third, the study is conducted in batch mode on a pre-processed dataset and does not address operation directly on server logs in real time.

1.2.2 Analysing HTTP requests with structural features (Althubiti et al., 2017)

Althubiti, Yuan, and Esterline [?] also classify HTTP requests on CSIC 2010, but under a different feature philosophy. Instead of a text representation, the authors use nine *structural*, numeric features proposed by Nguyen et al. [?], such as the length of the request, the length of the argument section, the number of arguments, the length of the path, and the number of special characters in the path. They employ the Weka toolkit with the `WrapperSubsetEval` attribute evaluator (using the J48 classifier and ten-fold cross-validation) combined with the `BestFirst` search strategy to select the five best features. On this compact feature set, most algorithms (Random Forest, Logistic Regression, J48, AdaBoost, SGD) achieve a very high detection rate of approximately 99.9%, with only Naïve Bayes performing markedly worse (around 88%).

The advantage of this work is a lightweight numeric feature set that is easy to compute, fast to train, and highly accurate; feature selection further reduces the training time while improving the results. However, the near-perfect detection figures also raise questions about generalisation when the training and test data share the same distribution, and the study remains entirely within the supervised paradigm with no mechanism for novel attacks. Moreover, although features based purely on length and counts are sensitive to buffer-overflow attacks, they may miss attacks that are semantically sophisticated yet not anomalous in length.

1.2.3 Comparing seven algorithms and a voting classifier (Rawashdeh, 2023)

Rawashdeh [?] extends the comparison to seven algorithms—Decision Tree, Random Forest, Gradient Boosting, XGBoost, AdaBoost, Multilayer Perceptron, and a Voting Classifier—on the Kaggle edition of CSIC 2010 (61,065 samples, 16 features corresponding to HTTP header fields). The non-numeric features are converted to numeric form through Label Encoding. The results show that the Voting Classifier and the Decision Tree deliver the best performance, with an accuracy of about 89%. A notable contribution of this

work is its demonstration of the benefit of *ensemble* learning through a voting mechanism, in which the final label is decided by the majority vote of the component classifiers.

The limitation of the study is that its overall accuracy ($\approx 89\%$) is lower than that of the two works above, partly because label-encoding categorical attributes does not preserve the semantics of the HTTP content. Furthermore, although a voting mechanism is used, all components are supervised models, so the system still inherits the common drawback of being unable to detect attack types lying outside the training distribution.

1.2.4 Deep learning and one-class approaches on the same dataset

Beyond classical classifiers, several studies surveyed in [?] apply deep learning to CSIC 2010. Kim et al. [?] combine a Convolutional Neural Network with a Long Short-Term Memory network (CNN with LSTM) and reach an accuracy of about 91.54%. Of particular relevance to this thesis, Vartouni et al. [?] use a Stacked Auto-Encoder—a reconstruction-based model with an *anomaly detection* flavour—and obtain an accuracy of about 88.32%. Betarte et al. [?] compare Random Forest, K-Nearest Neighbours, and Support Vector Machine, with Random Forest performing best in that group. These works reveal two trends: on the one hand, complex deep models do not necessarily outperform compact structural classifiers on CSIC 2010; on the other hand, the reconstruction-based anomaly direction opens up the possibility of detecting novel attacks without requiring attack labels during training.

1.2.5 Synthesis and research motivation

Table 1.1 summarises the representative studies just reviewed, contrasting them along the criteria of dataset, method, approach, and main limitation. Several important commonalities emerge from this table.

First, the vast majority of studies adopt a *single* approach, either purely supervised learning or purely anomaly detection. The supervised direction yields high accuracy on known attacks but requires labelled data and is weak against entirely new attacks. The anomaly direction has the potential to catch previously unseen attacks but typically suffers from a high false-alarm rate and poor stability across algorithms. Second, most studies work on offline pre-processed feature sets; few systems operate directly on raw server logs and handle practical situations such as multi-layer encoded payloads or attacks delivered through the `POST` body. Third, the signature-based detection layer—which has very high precision and good explainability—is often ignored or kept separate from the machine learning models rather than combined in a systematic way.

These observations are precisely the motivation of this thesis. The system’s own experiments confirm the complementarity of the approaches: the signature tier reaches a precision of 99.61% but a recall of only 24.07% (missing most attacks), whereas the unsupervised anomaly models attain high recall but widely varying precision across algorithms. Therefore, the thesis proposes a *three-tier hybrid* architecture that simultaneously combines (i) signature- and rule-based detection, (ii) supervised learning with Random Forest, and (iii) unsupervised anomaly detection following the one-class learning principle. A smart consensus decision mechanism lets the supervised tier validate the detections of the anomaly tier, thereby exploiting the high recall of anomaly detection while keeping false alarms under control. This is the core difference between this thesis and the surveyed studies.

Table 1.1: Summary of related work on web attack detection over CSIC 2010.

Study	Dataset	Representative method	Approach	Main limitation
Pham et al., 2016	CSIC 2010	LR, RF, DT, AdaBoost, SGD on TF-IDF features	Supervised, single model	Bulky features, no novel-attack handling
Nguyen et al., 2011	CSIC 2010, ECML/PKDD	GeFS feature selection (CFS, mRMR)	Supervised	Hand-crafted structural features
Althubiti et al., 2017	CSIC 2010	RF, LR, J48, AdaBoost, SGD, NB on 5 features	Supervised	Near-perfect scores, generalisation doubts
Rawashdeh, 2023	CSIC 2010 (Kaggle)	DT, RF, GB, XGB, AdaBoost, MLP, Voting	Supervised ensemble	Accuracy $\sim 89\%$, label encoding loses semantics
Kim et al., 2020	CSIC 2010	CNN + LSTM, DNN	Supervised deep learning	Complex, needs much labelled data
Vartouni et al., 2018	CSIC 2010	Stacked Auto-Encoder	Anomaly detection	Accuracy $\sim 88\%$, recall threshold-dependent

1.3 Intrusion Detection Systems

An Intrusion Detection System (IDS) is a security component whose task is to monitor the events occurring in a computer system or network and to analyse them for signs of intrusive behaviour [?]. According to the monitoring scope, an IDS is usually divided into a Network-based IDS, which monitors traffic crossing the network, and a Host-based IDS, which monitors activity on a specific machine. This thesis belongs to the web application intrusion detection category, a specialised form that operates on the application-layer logs of a web server.

According to the detection method, the classical literature distinguishes two main paradigms, summarised by Althubiti et al. [?] as follows. The first paradigm is *misuse detection*, also called signature-based detection, which operates by matching the current activity against known attack patterns, usually through pattern-matching algorithms. Its advantage is high accuracy and few false alarms for known attacks, but its inherent drawback is the inability to detect new attacks that are absent from the signature set. The second paradigm is *anomaly detection*, which studies the behaviour of the user or the system in order to build a model of “normal” behaviour, then treats significant deviations from this model as anomalies; this direction is typically based on machine learning techniques and is able to detect new attacks, at the cost of potentially generating more false alarms.

These two paradigms are clearly complementary: misuse detection is strong in precision while anomaly detection is strong in covering unknown situations. This observation is the theoretical foundation for the hybrid architecture of the thesis, in which the signature tier (Section 1.5) represents misuse detection and the unsupervised tier (Section 1.7) represents anomaly detection; the supervised tier (Section 1.6) plays an intermediary role, both generalising from known attacks and providing a confidence probability that

reconciles the two paradigms.

1.4 Common Web Application Attacks

To design appropriate detection rules and features, it is necessary to understand the nature of the web attacks that the system targets. Most of these attacks appear in the list of the ten most critical web application security risks published by the OWASP project [?]. In the OWASP Top 10 of 2021, the “Injection” group ranks at position A03 and includes several sub-types such as SQL injection, operating-system command injection, and cross-site scripting [?]. The attack types directly relevant to the thesis are presented below.

- (i) **SQL Injection.** The attack occurs when untrusted data is sent to an interpreter as part of a database query [?]. By inserting clauses such as `UNION SELECT`, `OR '1'='1`, or statement terminators, the attacker can trick the interpreter into executing unintended operations and thus access or modify data without authorisation.
- (ii) **Cross-Site Scripting (XSS).** An XSS flaw appears when an application takes untrusted data and sends it to a browser without proper validation or escaping [?]. The attacker can then execute script code in the victim’s browser to hijack the session, deface the page, or redirect the user to a malicious site.
- (iii) **Path Traversal and Local File Inclusion.** The attacker exploits the `../` sequence to escape the intended directory and access sensitive system files such as `/etc/passwd` or `win.ini`.
- (iv) **Command Injection.** The attack allows the execution of arbitrary operating-system commands through command-chaining characters (`;`, `|`, `&&`, the backtick) placed before dangerous commands such as `rm`, `cat`, `wget`, or `bash`.
- (v) **Remote File Inclusion.** An attack in which a parameter value is a remote URL (starting with `http://`, `https://`, or `ftp://`), aiming to force the server to load and execute code from an external source.
- (vi) **Sensitive file access, parameter tampering, and method tampering.** Sensitive file access is an attempt to read configuration or backup files such as `.env`, `wp-config.php`, or `.sql`. Parameter tampering is the modification of a parameter name relative to the legitimate names an endpoint usually accepts, for example changing `modo` to `modoA`. HTTP Method Tampering is the use of unusual methods such as `PUT`, `DELETE`, or `TRACE` on endpoints that normally serve only `GET/POST`.

Enumerating these attack classes helps the reader grasp the scope of the problem; how each one is actually detected—through dedicated regular expressions and rules—is detailed in the next chapter.

1.5 Signature-Based Detection with Regular Expressions

Signature-based detection is the technique of matching the content of a request against a set of known attack patterns. In the thesis, these patterns are expressed as *regular*

expressions (regex)—a formal language that allows a compact description of sets of character strings. Each attack group in Section 1.4 corresponds to a pre-compiled regular expression, for example `SQLI_RE` for SQL injection or `XSS_RE` for cross-site scripting. When a request is loaded, the `detect_rule_based` function applies these expressions to the payload in turn and returns the list of rules that fired.

An important challenge for signature-based detection is the *encoding evasion* technique, in which attackers encode the URL in several layers to hide the payload, so that matching on the raw string fails. To overcome this, the system iteratively URL-decodes the request up to three times (the `multi_decode` function and the decoding loop in `detect_rule_based`) to recover the original payload—for instance the tautology `'OR'a'='a'`—before matching it against the rules.

The advantage of the signature tier is its very high precision and its transparent explainability: every alert explicitly indicates which rule fired, helping the analyst quickly verify it. However, as noted, the inherent weakness of this direction is low recall—it catches only what has been explicitly described in the rule set. The thesis’s experimental results reflect exactly this property, with a precision of 99.61% but a recall of only 24.07%. This is precisely why the signature tier must be complemented by the machine learning tiers in the following sections.

1.6 Supervised Machine Learning for Attack Classification

Supervised learning is an approach in which the model learns from a labelled dataset, each sample consisting of a feature vector and a class label (here, *attack* or *benign*), in order to build a function that maps features to labels and predict for new data [?]. The thesis uses supervised learning as the second detection tier, with two algorithms implemented in `supervised_detector.py`: Random Forest (serving as the primary model) and Logistic Regression (serving as a comparison baseline).

1.6.1 Random Forest

The Random Forest, proposed by Breiman in 2001 [?], is an ensemble learning method that adds randomness to the bagging technique. Each decision tree in the forest is built from a different bootstrap sample of the data; moreover, at each node, instead of selecting the best split over all attributes, the algorithm considers only a randomly chosen subset of attributes [?]. This strategy, although seemingly counter-intuitive, helps the random forest achieve high performance and robustness against overfitting. For a classification task, the final result is the class voted for by the most trees.

Formally, when growing each tree, a node is split so as to minimise the impurity of the resulting child nodes. Writing p_i for the proportion of class i among the samples in a node, the two impurity measures commonly used are the Gini impurity and the Shannon entropy,

$$G = 1 - \sum_i p_i^2, \quad H = - \sum_i p_i \log_2 p_i. \quad (1.1)$$

For an ensemble of B trees $\{T_b\}_{b=1}^B$, the predicted class of a sample x is the majority vote $\hat{y}(x) = \text{mode}\{T_b(x)\}$, and the probability that x is an attack is estimated as the fraction

of trees voting for the attack class,

$$p_{\text{attack}}(x) = \frac{1}{B} \sum_{b=1}^B \mathbf{1}[T_b(x) = \text{attack}], \quad (1.2)$$

where $\mathbf{1}[\cdot]$ is the indicator function. This probability, rather than the hard label, is what the smart consensus mechanism later exploits.

In the thesis, the Random Forest is configured with 200 trees and balanced class weights (`class_weight="balanced"`); being threshold-based, it requires no feature scaling. Crucially, besides the predicted label it also returns the *probability* of the attack class through `predict_proba`. This probability is pivotal for the smart consensus mechanism, letting the supervised tier validate the unsupervised tier’s detections at different confidence levels rather than issuing a hard decision. In experiments the Random Forest attains an F1 score of about 94.78%, the best single model in the system.

1.6.2 Logistic Regression

Logistic Regression is a linear classifier that estimates the probability of a sample belonging to a class as a smooth function of the input features [?], trained by maximum likelihood. Concretely, for a weight vector $\mathbf{w}$ and bias b it models the attack probability with the logistic (sigmoid) function,

$$P(y = 1 \mid x) = \sigma(\mathbf{w}^\top x + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}, \quad (1.3)$$

and its parameters are fitted by minimising the binary cross-entropy (log-loss) over the n training samples,

$$\mathcal{L} = - \sum_{i=1}^n \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right], \quad p_i = P(y = 1 \mid x_i). \quad (1.4)$$

Pham et al. [?] note that it suits binary classification well and has low variance, so it is less prone to overfitting. In the thesis it serves as a backup and reference model; being scale-sensitive, it is applied with a feature-scaling step (`StandardScaler`), unlike the Random Forest. Its F1 score is about 79.99%, lower than the Random Forest, consistent with a linear model’s difficulty in capturing the non-linear interactions among the 22 features.

An important commonality of both supervised models in the thesis is that they use the *same 22-feature space* as the unsupervised tier (Section 1.8). Consequently, the performance difference between the two tiers reflects the genuine “labelled versus unlabelled” distinction rather than being confounded by differences in feature representation.

1.7 Unsupervised Anomaly Detection via One-Class Learning

The third detection tier of the thesis follows the unsupervised direction in order to catch new attacks that the supervised tier has never learned. The core principle here is *one-class learning*: the model is trained only on “normal” data to learn the distribution of clean traffic, then treats any significant deviation from that distribution as an anomaly,

that is, an attack [?]. This approach requires no attack labels during training, which fits the reality that attack traffic is both rare and difficult to collect comprehensively.

An important methodological point that the thesis strictly observes is to *train the third tier only on clean data* (34,941 benign log lines), rather than on a mixture of attack and benign traffic. Experiments show that with incorrect training (on a mixture containing 50% attacks), the F1 score of the models reaches only about 33%; with correct training following the one-class principle on clean data, the F1 score rises sharply, for example the Local Outlier Factor model attains 91.57%. This improvement confirms that one-class learning is not merely a technical choice but a precondition for the unsupervised tier to function correctly. The thesis implements three complementary algorithms, described below.

1.7.1 Local Outlier Factor (LOF)

The Local Outlier Factor, proposed by Breunig et al. in 2000 [?], is an outlier-detection algorithm based on *local density*. For each data point, LOF estimates the local density from the distances to its k nearest neighbours, then compares the density of that point with the density of its neighbours. A point lying in a region markedly sparser than its neighbourhood receives a high LOF score and is treated as an outlier. The “local” nature of the algorithm enables it to detect subtle anomalies sitting between regions of different density, which methods based on a global threshold tend to miss.

Formally, let $N_k(p)$ be the set of the k nearest neighbours of a point p and let $d(p, o)$ be the distance between p and o . The *reachability distance* of p from o smooths short distances using the k -distance of o ,

$$\text{reach-dist}_k(p, o) = \max\{k\text{-distance}(o), d(p, o)\}. \quad (1.5)$$

The *local reachability density* of p is the inverse of the average reachability distance to its neighbours,

$$\text{lrd}_k(p) = \left(\frac{1}{|N_k(p)|} \sum_{o \in N_k(p)} \text{reach-dist}_k(p, o) \right)^{-1}, \quad (1.6)$$

and the Local Outlier Factor of p is the average ratio of the neighbours’ densities to its own,

$$\text{LOF}_k(p) = \frac{1}{|N_k(p)|} \sum_{o \in N_k(p)} \frac{\text{lrd}_k(o)}{\text{lrd}_k(p)}. \quad (1.7)$$

A value of $\text{LOF}_k(p) \approx 1$ indicates that p has a density comparable to its neighbours (an inlier), whereas $\text{LOF}_k(p) \gg 1$ marks p as an outlier sitting in a comparatively sparse region. In the thesis, LOF is configured with `n_neighbors = 20` and the novelty-detection mode (`novelty = True`) so that it can predict on unseen data. It is the best-performing unsupervised model, with a precision of 88.28% and a recall of 95.13%, and is therefore chosen as the default third tier of the system.

1.7.2 Isolation Forest

The Isolation Forest, proposed by Liu, Ting, and Zhou in 2008 [?], approaches the problem from a different angle: instead of profiling the normal points, the algorithm directly *isolates* the anomalous points. The idea rests on two properties of outliers: they are the

minority and they have attribute values markedly different from the bulk of the data. By recursively and randomly partitioning the feature space, anomalous points tend to be isolated after fewer splits, corresponding to a smaller average depth in the tree.

Quantitatively, let $h(x)$ be the path length of a point x (the number of edges from the root to the leaf isolating it) and let $E(h(x))$ be its average over all trees. The expected path length of an unsuccessful search in a binary tree of n points is used for normalisation,

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n}, \quad H(i) \approx \ln(i) + 0.5772, \quad (1.8)$$

where 0.5772 is the Euler–Mascheroni constant. The anomaly score is then

$$s(x, n) = 2^{-E(h(x))/c(n)}, \quad (1.9)$$

so that $s \rightarrow 1$ when the average path length is very short (a likely anomaly), and $s \approx 0.5$ when it is comparable to that of a normal point. The advantage of the Isolation Forest is its high speed and low memory demand thanks to its use of sampling. In the thesis, this model attains high recall (93.14%) but a lower precision than LOF (63.02%), so it serves as a comparison model rather than the operational choice.

1.7.3 One-Class SVM

The One-Class SVM, proposed by Schölkopf et al. [?], extends the support vector machine to the novelty-detection problem. The algorithm learns a boundary enclosing the region of normal data in the feature space; points falling outside this boundary are considered anomalous. Mapping the inputs into a feature space through a function $\phi(\cdot)$, it solves the optimisation problem

$$\min_{\mathbf{w}, \xi, \rho} \frac{1}{2} \|\mathbf{w}\|^2 + \frac{1}{\nu n} \sum_{i=1}^n \xi_i - \rho \quad \text{s.t.} \quad \langle \mathbf{w}, \phi(x_i) \rangle \geq \rho - \xi_i, \quad \xi_i \geq 0, \quad (1.10)$$

and classifies a new point by the sign of the decision function $f(x) = \text{sign}(\langle \mathbf{w}, \phi(x) \rangle - \rho)$, a negative sign indicating an anomaly. The parameter $\nu \in (0, 1]$ upper-bounds the fraction of training points allowed to fall outside the boundary. By using a kernel function—typically the radial basis function (RBF) kernel $K(x, x') = \exp(-\gamma \|x - x'\|^2)$ as in the thesis—the One-Class SVM can describe complex non-linear boundaries. Because the algorithm is sensitive to feature scale, the thesis applies a scaling step before training. The model attains a recall of 91.38% but a precision of only 62.27%, similar to the Isolation Forest, reinforcing the conclusion that among the three one-class algorithms, LOF is the most suitable for the HTTP log data of this problem.

1.8 Feature Extraction and Evaluation Metrics

1.8.1 Feature extraction from HTTP requests

Because machine learning algorithms require numeric input, an essential step is to transform each HTTP request into a feature vector. The thesis does not use a bulky bag-of-words representation as in [?] but instead designs a compact 22-dimensional vector that is shared by both the supervised and the unsupervised tiers. Conceptually, these

features fall into three groups: structural features (length, path depth, parameter count, and similar size measures), randomness-and-encoding features (the Shannon entropy of the decoded payload and the number of encoding layers, which expose obfuscation and evasion), and semantic-and-contextual features (the signature score, the risk-keyword density, the suspicion of the **User-Agent** and **Referer** headers, and the per-endpoint unknown-parameter ratio). Using a shared feature space ensures consistency and enables a fair comparison between the approaches, while keeping the model compact and largely independent of the vocabulary of any specific application. The precise composition of the vector and the extraction procedure are presented in the next chapter.

1.8.2 Evaluation metrics

The performance of a detection system is usually assessed through the confusion matrix, which comprises four quantities: the number of true positives (TP, attacks detected correctly), true negatives (TN, clean traffic recognised correctly), false positives (FP, clean traffic wrongly flagged as an attack), and false negatives (FN, attacks that are missed). From these four quantities, the thesis uses the following standard metrics, defined consistently with Althubiti et al. [?].

Precision is the proportion of attack predictions that are actually correct, reflecting how “clean” the alerts are:

$$\text{Precision} = \frac{TP}{TP + FP}. \quad (1.11)$$

Recall is the proportion of actual attacks that the system catches, reflecting the degree of missed detections:

$$\text{Recall} = \frac{TP}{TP + FN}. \quad (1.12)$$

The *F1 score* is the harmonic mean of precision and recall, providing a single number that balances the two aspects:

$$F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (1.13)$$

In addition, the thesis emphasises the *F2 score* as a metric particularly suited to the intrusion detection problem:

$$F_2 = \frac{5 \times \text{Precision} \times \text{Recall}}{4 \times \text{Precision} + \text{Recall}}. \quad (1.14)$$

The F2 score is a variant of the F-score family in which recall is weighted twice as heavily as precision. This choice stems from the security context: missing an attack (a false negative) may lead to a serious data breach, whereas a false alarm (a false positive) costs the analyst only a few minutes of investigation. Therefore, in the IDS context, prioritising recall is reasonable, and the F2 score reflects that priority better than the F1 score. The `_safe_metrics` function in `apache_log.py` computes all of the above metrics simultaneously for every model configuration, forming the basis for the comparison table and the evaluation charts in the subsequent chapters.

1.9 Chapter Summary

This chapter has established the full foundation for moving on to the design and implementation of the system. Section 1.1 delimited the problem: detecting application-layer

web attacks from Apache logs, with a concrete catalogue of in-scope attack classes and a set of deliberately excluded limitations. Section 1.2 surveyed and critically analysed a range of representative studies on the CSIC 2010 dataset, showing that most research pursues a single approach and rarely combines signature-based detection with machine learning; this gap is precisely the motivation for the three-tier hybrid architecture of the thesis. Sections 1.3 through 1.8 presented the foundational knowledge tightly bound to the system: the notion of an IDS and its two detection paradigms, web attack types according to OWASP, the regular-expression-based signature technique together with its anti-evasion decoding mechanism, supervised learning algorithms (Random Forest, Logistic Regression) and unsupervised one-class learning algorithms (LOF, Isolation Forest, One-Class SVM), as well as the shared 22-feature extraction and the evaluation metrics centred on the F2 score. On the basis of this knowledge and analysis, the next chapter will present the detailed design of the three-tier architecture together with the smart consensus decision mechanism, turning the theoretical principles laid out here into a complete detection system.